

# AI-Powered Interview Preparation Platform

## 6-Week Sprint Execution Plan

Angular | FastAPI (Python) | PostgreSQL | OpenAI / Gemini

### Project Overview

A full-stack web application that helps students prepare for technical and HR interviews through AI-driven mock sessions, structured feedback, and visual progress analytics.

### Tech Stack

| Layer          | Technology                      | Purpose                                           |
|----------------|---------------------------------|---------------------------------------------------|
| Frontend       | Angular 17+                     | SPA, routing, lazy modules, standalone components |
| Styling        | Angular Material + Tailwind CSS | Responsive UI, dark mode, design system           |
| Charts         | ng2-charts (Chart.js)           | Line, bar, radar charts on dashboard              |
| Backend        | FastAPI (Python 3.11)           | REST APIs, async endpoints, Pydantic validation   |
| Database       | PostgreSQL 15                   | Relational storage for users, sessions, questions |
| ORM            | SQLAlchemy + Alembic            | ORM models and database migrations                |
| Authentication | JWT + bcrypt + OAuth2           | Secure sessions, refresh tokens, Google login     |
| AI / NLP       | OpenAI GPT-4 / Gemini           | Answer scoring, feedback, model answers           |
| Task Queue     | Celery + Redis                  | Async AI jobs, scheduled reminders                |
| File Storage   | AWS S3 / Cloudinary             | Resume uploads, PDF exports                       |
| Deployment     | Docker + Nginx                  | Containerized services, reverse proxy             |
| CI/CD          | GitHub Actions                  | Automated test and deployment pipeline            |

### Sprint-wise Execution Plan

|        |                                                                                        |
|--------|----------------------------------------------------------------------------------------|
| Week 1 | <b>Foundation &amp; Authentication</b><br>Project setup, User auth, Profile management |
|--------|----------------------------------------------------------------------------------------|

### Backend Tasks

| Type | Task                                                                        |
|------|-----------------------------------------------------------------------------|
| BE   | FastAPI project init, folder structure (routers, models, schemas, services) |
| DB   | PostgreSQL setup, SQLAlchemy ORM configuration                              |
| DB   | Create tables: users, profiles, sessions — run Alembic migrations           |
| BE   | Register & Login API with bcrypt password hashing                           |
| BE   | JWT access token + refresh token logic                                      |
| BE   | Google OAuth2 sign-in integration                                           |
| BE   | Password reset via email (SMTP / SendGrid)                                  |

|    |                                                                                |
|----|--------------------------------------------------------------------------------|
| BE | Profile CRUD — target companies, preferred interview types, resume upload (S3) |
|----|--------------------------------------------------------------------------------|

## Frontend Tasks

| Type | Task                                                            |
|------|-----------------------------------------------------------------|
| FE   | Angular project init with routing, lazy-loaded modules          |
| FE   | Angular Material + Tailwind CSS configuration                   |
| FE   | Auth module: Register, Login, Forgot Password pages             |
| FE   | JWT HTTP Interceptor + AuthGuard for protected routes           |
| FE   | Profile page: edit info, upload resume, select target companies |
| FE   | App shell layout: sidebar navigation + top navbar               |

## Week 1 Deliverables

- Working register/login with JWT and refresh token
- Google OAuth2 sign-in functional
- Profile page with resume upload to S3/Cloudinary
- Protected routes with AuthGuard

**Week 2****Question Bank & Admin Panel**

Content foundation, search/filter, admin CRUD

**Backend Tasks**

| Type | Task                                                                                      |
|------|-------------------------------------------------------------------------------------------|
| DB   | Create tables: questions, companies, topics, difficulty_levels, question_types, bookmarks |
| BE   | Question CRUD APIs — add, edit, delete, get (admin protected)                             |
| BE   | Filter API: by company, topic, difficulty, question type                                  |
| BE   | Full-text search API using PostgreSQL tsvector                                            |
| BE   | Bookmark question API (per user, toggle)                                                  |
| BE   | Bulk import from CSV / JSON file (admin only)                                             |
| BE   | Admin role middleware — role-based access control                                         |
| BE   | Audit log table + logging for all admin actions                                           |

**Frontend Tasks**

| Type | Task                                                                         |
|------|------------------------------------------------------------------------------|
| FE   | Question bank list page with filters (company, topic, difficulty) and search |
| FE   | Question detail view with bookmark toggle button                             |
| FE   | Admin panel: question management table (add, edit, delete)                   |
| FE   | Admin: bulk import UI with drag-and-drop CSV/JSON upload                     |
| FE   | Company profiles pages: interview format, rounds, tips                       |

**Week 2 Deliverables**

- Question bank with full filter, search, and bookmark functionality
- Admin panel with complete CRUD and bulk import
- Company profile pages with targeted preparation content

**Week 3****Mock Interview Engine**

Core interview session flow, timer, pause/resume

**Backend Tasks**

| Type | Task                                                                           |
|------|--------------------------------------------------------------------------------|
| DB   | Create tables: interview_sessions, session_answers, session_configs            |
| BE   | Create session API — accepts company, interview type, duration, question count |
| BE   | Question-serving logic: randomised selection, no repeats in same session       |
| BE   | Save answer API — stores answer text + time taken per question                 |
| BE   | Pause / Resume session API with server-side state persistence                  |
| BE   | Complete session API — marks session done, triggers AI feedback job            |
| BE   | Session history list API — paginated list per user                             |

**Frontend Tasks**

| Type | Task                                                                  |
|------|-----------------------------------------------------------------------|
| FE   | Session config screen: select company, type, duration, question count |
| FE   | Interview screen: question display + answer textarea                  |
| FE   | Countdown timer component (per question, configurable duration)       |
| FE   | Progress indicator: questions answered vs total                       |
| FE   | Pause/resume overlay with confirmation dialog                         |
| FE   | Session complete screen with loading state (waiting for AI feedback)  |
| FE   | Session history page: list of past sessions with dates and scores     |

### Week 3 Deliverables

- Complete end-to-end interview session flow
- Countdown timer, pause/resume, auto-complete on timeout
- Session history visible per user

**Week 4****AI Feedback & Scoring Engine**

OpenAI/Gemini integration, async Celery queue

**Backend Tasks**

| Type | Task                                                                       |
|------|----------------------------------------------------------------------------|
| AI   | OpenAI GPT-4 / Gemini API client setup with API key management             |
| AI   | Prompt engineering: evaluate correctness, clarity, structure, completeness |
| AI   | Response parser: extract score/10, strengths, areas to improve             |
| AI   | Model answer generator — generate ideal answer per question                |
| BE   | Celery + Redis setup for async AI feedback job queue                       |
| DB   | Create tables: feedback_reports, answer_feedbacks                          |
| BE   | Session report API: overall score, topic breakdown, improvement tips       |
| BE   | Admin: view and resolve student-reported AI feedback issues                |

**Frontend Tasks**

| Type | Task                                                              |
|------|-------------------------------------------------------------------|
| FE   | Feedback report page per session (accessible from history)        |
| FE   | Per-answer scorecard: score bar, strengths list, improvement tips |
| FE   | Model answer reveal toggle (hidden by default)                    |
| FE   | Overall session summary card with total score and grade           |
| FE   | Polling/loading state while Celery processes AI feedback          |
| FE   | Flag/report issue button on feedback page (sends to admin)        |

**Week 4 Deliverables**

- AI scoring pipeline working with async Celery queue (no HTTP timeouts)
- Detailed per-answer feedback report with scores and suggestions
- Model answers and personalized improvement tips per session

**Week 5****Dashboard, Analytics & Notifications**

Charts, progress tracking, company readiness

**Backend Tasks**

| Type | Task                                                                         |
|------|------------------------------------------------------------------------------|
| BE   | Score trend API — scores over time by date range filter                      |
| BE   | Topic-wise performance breakdown API                                         |
| BE   | Weakest areas detection + practice streak calculation                        |
| BE   | Company readiness score API (per company, based on history)                  |
| BE   | Company recommendation engine — match companies to user skill level          |
| BE   | PDF export API for performance report (ReportLab / WeasyPrint)               |
| BE   | Celery Beat: daily practice reminder scheduler                               |
| BE   | Notifications: email (SMTP) + in-app milestone alerts (100 Qs, 7-day streak) |

## Frontend Tasks

| Type | Task                                                                   |
|------|------------------------------------------------------------------------|
| FE   | Main dashboard: summary stat cards (total sessions, avg score, streak) |
| FE   | Score trend line chart using ng2-charts (Chart.js)                     |
| FE   | Topic performance radar/bar chart (topic breakdown)                    |
| FE   | Practice streak calendar heatmap                                       |
| FE   | Company readiness cards with progress bars                             |
| FE   | Download PDF performance report button                                 |
| FE   | Notification centre page + bell icon with unread badge                 |
| FE   | Daily goal settings in user preferences                                |

## Week 5 Deliverables

- Interactive analytics dashboard with all charts live
- Company readiness scores + smart recommendations
- Email and in-app notifications with milestone alerts

## Week 6

## Testing, Polish & Deployment

QA, Docker, CI/CD, production go-live

### Testing Tasks

| Type | Task                                                                |
|------|---------------------------------------------------------------------|
| TEST | Pytest: unit tests for all FastAPI route handlers and services      |
| TEST | Pytest: AI feedback scoring validation with mock API responses      |
| TEST | Angular Karma + Jasmine: component unit tests for key modules       |
| TEST | E2E tests: full session creation → completion → feedback flow       |
| TEST | Celery queue stress test: multiple concurrent AI feedback jobs      |
| TEST | Cross-browser testing (Chrome, Firefox, Safari) + mobile responsive |

### Deployment Tasks

| Type | Task                                                                   |
|------|------------------------------------------------------------------------|
| BE   | Dockerfile for FastAPI app + separate Dockerfile for Celery workers    |
| FE   | Dockerfile for Angular with Nginx static file serving                  |
| BE   | docker-compose.yml orchestrating all services (app, worker, db, redis) |
| BE   | GitHub Actions: CI pipeline — lint, test on every PR                   |
| BE   | GitHub Actions: CD pipeline — auto deploy to server on main merge      |
| BE   | Nginx reverse proxy config — route /api to FastAPI, / to Angular       |
| BE   | .env file structure, secrets management (do not commit to git)         |
| BE   | AWS S3 / Cloudinary integration for resume and file storage            |

### Week 6 Deliverables

- All modules fully tested, critical bugs resolved
- Dockerized application deployed with CI/CD pipeline running
- Production-ready platform live and accessible

## Important Notes

### OpenAI API — Quota Management

Always use Celery async queue for AI feedback calls. Never call OpenAI directly from a synchronous FastAPI endpoint — it will timeout. Store your API key in .env and never hardcode it.

### Folder Structure (Brief)

#### FastAPI Backend

backend/

app/routers/ — auth.py, questions.py, sessions.py, feedback.py, admin.py

app/models/ — user.py, question.py, session.py, feedback.py

app/schemas/ — Pydantic request/response schemas

app/services/ — ai\_feedback.py, auth\_service.py, email.py

app/tasks/ — celery\_tasks.py (async AI jobs)  
alembic/ — database migration files

## Angular Frontend

frontend/src/app/  
modules/auth/ — login, register, forgot-password  
modules/question-bank/ — list, detail, bookmark  
modules/interview/ — config, session, complete  
modules/feedback/ — report, answer-card  
modules/dashboard/ — charts, stats, streaks  
modules/admin/ — question-crud, bulk-import  
core/interceptors/ — auth.interceptor.ts  
core/guards/ — auth.guard.ts

## Key Tips

- Start with FastAPI + PostgreSQL locally, then connect Angular after auth is working
- Use Alembic for all DB changes — never modify tables manually in production
- Keep AI prompts in a separate prompts.py file so they are easy to tune
- Test Celery locally with Redis running via Docker before Week 4
- Use environment variables for all secrets — OpenAI key, DB URL, JWT secret
- Angular lazy-loaded modules will keep bundle size small from day one